

Lab Assignment

International Cybersecurity and Digital Forensics Academy

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

1. Purpose of the Lab Assignment

This lab is designed to help students understand how HTTP traffic works in a network. It shows how a browser requests a webpage and its embedded image separately. Wireshark is used to capture and examine these network activities and the lab also teaches how to recover an image from captured network traffic.

2. Lab Report Structure

The lab report explains the work carried out during the practical lab which covers the purpose, objectives, tools, evidence collection and analysis. It also explains the HTTP requests and TCP packet information observed. The report describes how the embedded image was extracted and verified.

2.1. Title Page

- Lab Title: LAB 2: HTTP Analysis Using Wireshark - Embedded Image Traffic
- Student Name: Olanrewaju Wale Mustapha Oladimeji.
- Student ID: 2025/FWSD/11320.
- Course Name: SBT-DF203: Basic Networking Skills for Digital Forensics
- Instructor Name: Aminu Idris
- Date of Submission: 10/09/2026
- Version:

2.2. Executive Summary

The lab involved analyzing a webpage and an image transferred through HTTP. Wireshark was used to capture and examine the network traffic. The browser generated separate requests for the webpage and embedded image. The image data was carried through TCP and reconstructed during analysis.

2.3. Lab Objectives

The main objective was to understand how HTTP requests and responses work. The lab also aimed to identify HTML and image requests made by a browser. Another objective was to understand TCP segmentation and reassembly. The exercise also taught how to extract transferred objects from Wireshark.

2.4. Tools and Resources Used

Kali Linux : was used as the main forensic working environment.

Apache2 :was used to create and serve the local training webpage.

Wireshark: was used to capture and analyze network packets.

Curl : was used to generate controlled HTTP requests.

2.5. Methodology

Explain why one webpage can generate multiple HTTP requests and responses.

One webpage can generate multiple HTTP requests because it contains different resources. The browser first requests the main HTML page from the web server, It then requests embedded files such as images, CSS, and JavaScript separately.

Each requested file receives its own HTTP response from the server.

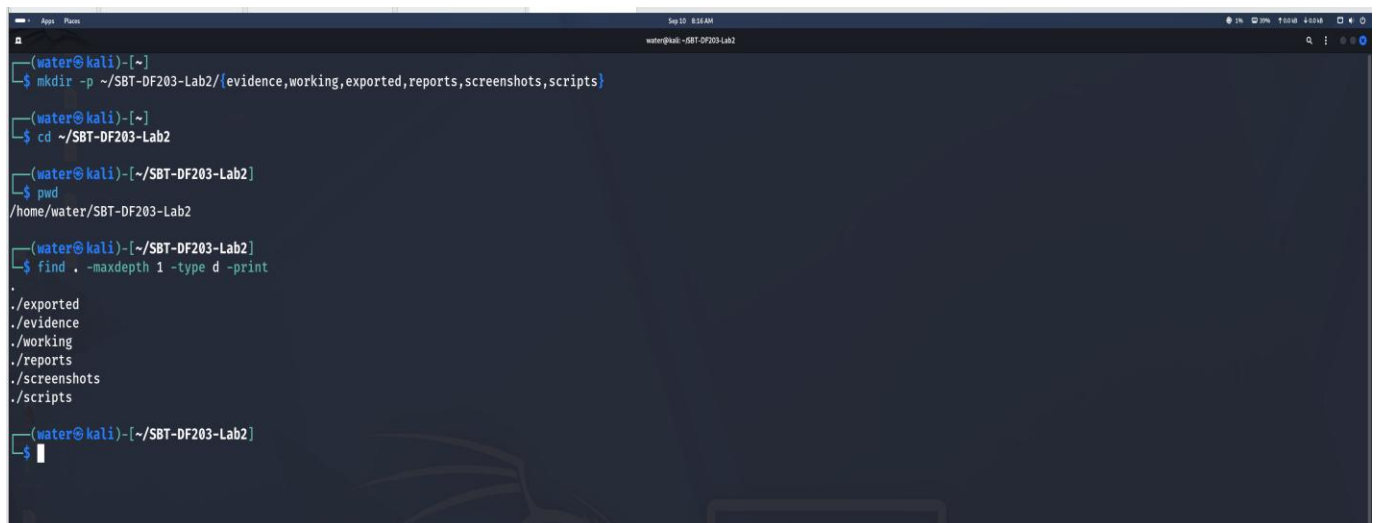
- **Structure and Evidence Preparation**

```
mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
```

```
cd ~/SBT-DF203-Lab2
```

```
pwd
```

```
find . -maxdepth 1 -type d -print
```



```
water@kali:~$ mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
water@kali:~$ cd ~/SBT-DF203-Lab2
water@kali:~/SBT-DF203-Lab2$ pwd
/home/water/SBT-DF203-Lab2
water@kali:~/SBT-DF203-Lab2$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./reports
./screenshots
./scripts
water@kali:~/SBT-DF203-Lab2$
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

sudo apt update

sudo apt install -y apache2 curl wireshark tshark imagemagick

sudo systemctl enable --now apache2

```
(water@kali)-[~]
└─$ sudo apt update
[sudo] password for water:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.7 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [261 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 76.7 MB in 50s (1,521 kB/s)
292 packages can be upgraded. Run 'apt list --upgradable' to see them.
apache2 is already the newest version (2.4.68-1).
curl is already the newest version (8.21.0-2).
curl set to manually installed.
wireshark is already the newest version (4.6.6-1).
wireshark set to manually installed.
tshark is already the newest version (4.6.6-1).
tshark set to manually installed.
imagemagick is already the newest version (8:7.1.2.27+dfsg1-1+b1).
imagemagick set to manually installed.
```

```
(water@kali)-[~]
└─$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 08:45:04 EDT; 4min 10s ago
     Invocation: 1f086f0439b478b884bdec34fd076
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 12747 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 7 (limit: 4403)
       Memory: 41.4M (peak: 41.7M)
          CPU: 155ms
    CGroup: /system.slice/apache2.service
            └─12747 /usr/sbin/apache2 -k start -DFOREGROUND
              └─12771 /usr/sbin/apache2 -k start -DFOREGROUND
                └─12772 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─12773 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─12774 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─12775 /usr/sbin/apache2 -k start -DFOREGROUND
                        └─12776 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 10 08:45:04 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 10 08:45:04 kali apache2[12747]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Sep 10 08:45:04 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-22/22 (END)
```

cp /PATH/TO/YOUR/PHOTO.jpg /var/www/html/lab_photo.jpg

```
(water@kali)-[~/Downloads]
└─$ ls
'120M (3)' '120M (4)' 120M.7z cacert.der output photo.jpg scalpel-output _tower_original_image_for_lab.bmp usb_fat_carving.001 usb_fat_carving.001.txt xampp-linux-x64-8.2.4-0-installer.run

(water@kali)-[~/Downloads]
└─$ sudo cp ~/Downloads/photo.jpg /var/www/html/lab_photo.jpg
[sudo] password for water:

(water@kali)-[~/Downloads]
└─$ ls -lh /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 12K Sep 10 09:04 /var/www/html/lab_photo.jpg

(water@kali)-[~/Downloads]
└─$
```

printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: YOUR NAME</p></body></html>\n' | sudo tee /var/www/html/image.html

```
water@kali: ~  
$ printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: water</p></body></html>\n' | sudo tee /var/www/html/image.html  
[sudo] password for water:  
<!DOCTYPE html>  
<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: water</p></body></html>  
water@kali: ~  
$
```

```
water@kali: ~  
$ cat /var/www/html/image.html  
<!DOCTYPE html>  
<html><body><h1>SBT-DF203 Image Traffic</h1><p>Analyst: water</p></body></html>  
water@kali: ~  
$
```

sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee
reports/source_object_hashes.txt

```
water@kali: ~  
$ sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee reports/source_object_hashes.txt  
tee: reports/source_object_hashes.txt: No such file or directory  
4a04afaacf4f4e3d990b3a573cc4d735f7d150583f11b8177e0270101060d488 /var/www/html/image.html  
e75e64610bcc9b70c65e520b7ee1d672c4946e9a3b52a7b234980dab8c42daa8 /var/www/html/lab_photo.jpg  
water@kali: ~  
$
```

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab2-Olanrewaju Wale (working directory: ~/SBT-DF203-Lab2)
Trainee name	water Olanrewaju Wale logged in as user "water" on host "kali"
Date and time started	Thu, Sep 10, 2026 – 08:16 AM
Evidence file name(s)	/var/www/html/image.html and /var/www/html/lab_photo.jpg, stored under the ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts} structure created for the lab
Source or generation method	lab_photo.jpg was copied with “sudo cp ~/Downloads/photo.jpg /var/www/html/lab_photo.jpg” . image.html was generated with a printf heredoc piped to “sudo tee /var/www/html/image.html”, embedding an tag.

Original SHA-256	image.html: 4a04afaacf4f4e3d990b3a573cc4d735f7d150583f11b8177e0270101060d488. lab_photo.jpg: e75e64610bcc9b70c65e520b7ee1d672c4946e9a3b52a7b234980dab8c42daa8
Working-copy SHA-256	
Analysis workstation/VM	Kali Linux host “kali”, user “water”, with apache2 2.4.68, curl 8.21.0, wireshark 4.6.6, tshark, and imagemagick installed; Apache serving locally at 127.0.0.1.
Notes on any changes	

8. Part A - Verify the Webpage and Eliminate Cache Effects

1. Use a non-sensitive image you are authorized to submit.
2. Open a private/incognito browser window or clear the cache before capture.
3. Confirm that image.html displays both text and the embedded image.
4. Record file type, dimensions and size of the source image.

file /var/www/html/lab_photo.jpg

identify /var/www/html/lab_photo.jpg

ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg

curl -I http://127.0.0.1/image.html

curl -I http://127.0.0.1/lab_photo.jpg

```

(water@kali)-[~]
$ ls -lh /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 12K Sep 10 09:04 /var/www/html/lab_photo.jpg
(water@kali)-[~]
$

```

identify /var/www/html/lab_photo.jpg

```

(water@kali)-[~]
$ identify /var/www/html/lab_photo.jpg
/var/www/html/lab_photo.jpg JPEG 269x187 269x187+0+0 8-bit sRGB 11615B 0.000u 0:00.000
(water@kali)-[~]
$

```

ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg

```
(water@kali)-[~]
$ ls -lh /var/www/html/image.html /var/www/html/lab_photo.jpg
-rw-r--r-- 1 root root 142 Sep 10 09:09 /var/www/html/image.html
-rw-r--r-- 1 root root 12K Sep 10 09:04 /var/www/html/lab_photo.jpg

(water@kali)-[~]
$
```

curl -I http://127.0.0.1/image.html

```
(water@kali)-[~]
$ curl -I http://127.0.0.1/image.html
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 13:30:31 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 13:09:49 GMT
ETag: "8e-65b20aec666ac"
Accept-Ranges: bytes
Content-Length: 142
Vary: Accept-Encoding
Content-Type: text/html

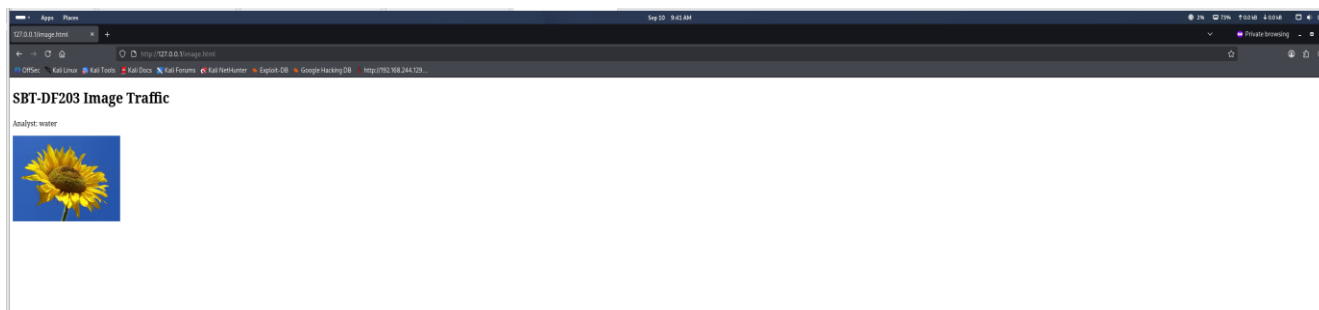
(water@kali)-[~]
$
```

curl -I http://127.0.0.1/lab_photo.jpg

```
(water@kali)-[~]
$ curl -I http://127.0.0.1/lab_photo.jpg
HTTP/1.1 200 OK
Date: Thu, 10 Sep 2026 13:32:31 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 13:04:38 GMT
ETag: "2d5f-65b209c36fa12"
Accept-Ranges: bytes
Content-Length: 11615
Content-Type: image/jpeg

(water@kali)-[~]
$
```

http://127.0.0.1/image.html



The webpage image.html was successfully opened from the local web server using http://127.0.0.1/image.html. The page displayed both the required text and the embedded lab_photo.jpg image. The image file type, dimensions, and file size were checked using the identify and ls -lh commands. The curl -I commands returned HTTP 200 OK for both the webpage and image, confirming that both resources were successfully served. A private browser window was used to reduce the possibility of cached content affecting the capture.

9. Part B - Capture Browser-Generated HTTP Traffic

Terminal 1

```
sudo tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
```

Browser

Open a private window and visit: <http://127.0.0.1/image.html>

Wait for the image to load, then stop the capture with Ctrl+C.

```
cp --preserve=timestamps evidence/image_traffic.pcapng
```

```
working/image_traffic_working.pcapng
```

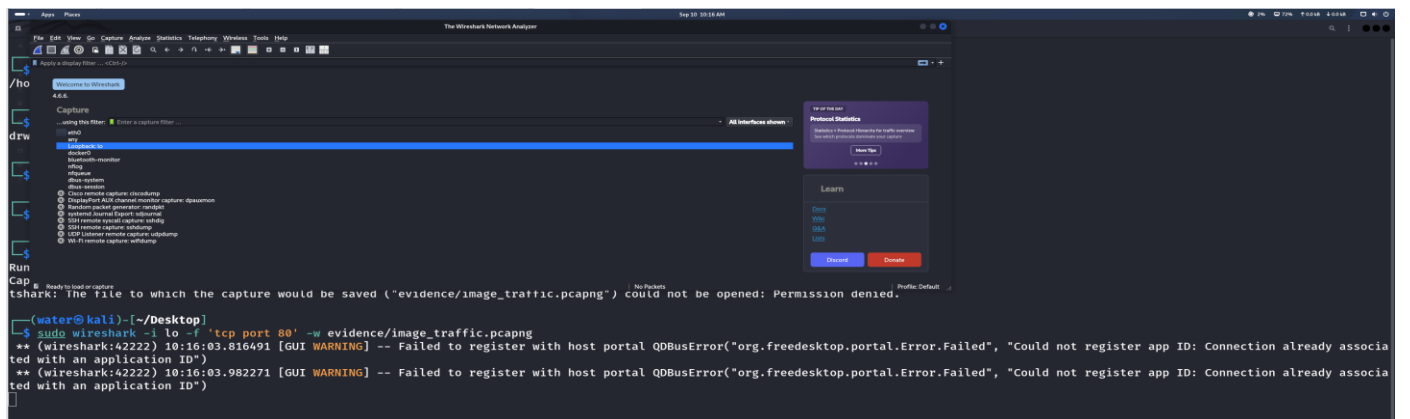
```
sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee  
reports/capture_hashes.txt
```

```
(water@kali)-[~/Desktop]
└─$ mkdir -p evidence working reports

(water@kali)-[~/Desktop]
└─$ ls
120M  CIP-B101_Lab1C_wale_water  image_hex.txt  lab  output  steg  usb
120M.7z  coded_again.jpg  jpeg_strings.txt  'lab file'  reconstructed.jpg  _tower_original_image_for_lab.bmp  usb_fat_carving.001
asecret.txt  evidence  _ub_low.jpg  note.txt  reports  secret.txt  _tower_original_image_for_lab.bmp.out  usb_fat_carving.001.txt
asecret.txt  File_carving.docx  knock-env  NTFS.zip  unnote.txt  working
```

Terminal 1

```
sudo wireshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
```

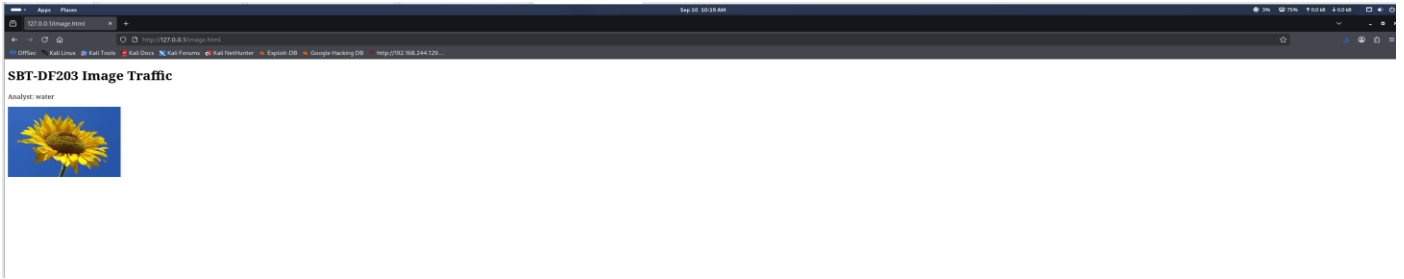


```
(water@kali)-[~/Desktop]
└─$ sudo wireshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
** (wireshark:42222) 10:16:03.816491 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
** (wireshark:42222) 10:16:03.982271 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
```

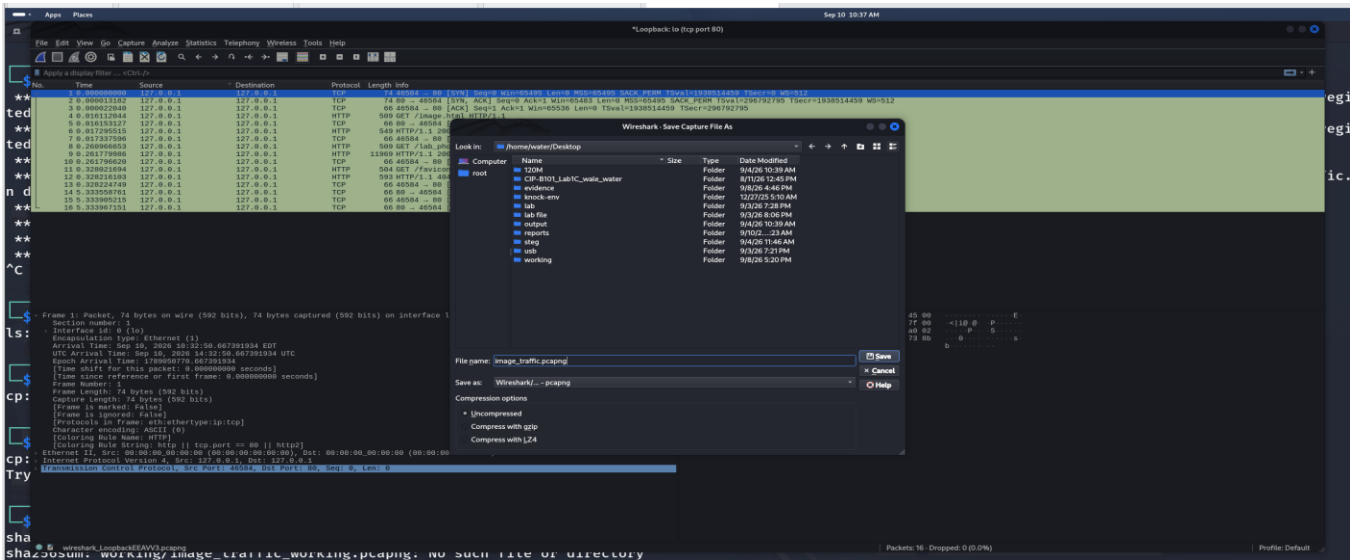
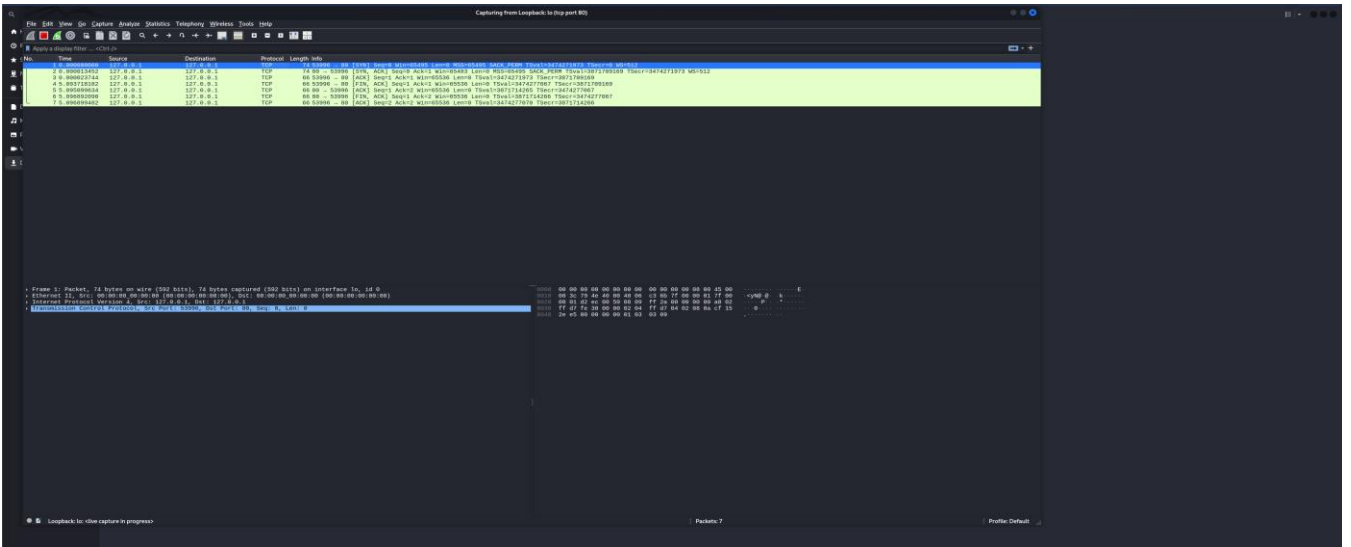
Browser

Open a private window and visit: <http://127.0.0.1/image.html>

Wait for the image to load, then stop the capture with Ctrl+C.



```
(water@kali) ~/Desktop
$ sudo wireshark -i lo -f 'tcp port 80' -w evidence
** (Wireshark:48818) 10:31:59.154158 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
** (Wireshark:48818) 10:32:18.206975 [Capture MESSAGE] -- Capture Start ...
** (Wireshark:48818) 10:32:18.252094 [Capture MESSAGE] -- Error message from child: "The file to which the capture would be saved ("evidence") could not be opened: Is a directory.", ""
** (Wireshark:48818) 10:32:20.720652 [Capture MESSAGE] -- Capture stopped.
** (Wireshark:48818) 10:32:22.950501 [Capture MESSAGE] -- Capture Start ...
** (Wireshark:48818) 10:32:22.987755 [Capture MESSAGE] -- Capture started
** (Wireshark:48818) 10:32:22.987788 [Capture MESSAGE] -- File: "/tmp/wireshark_LoopbackEAVV3.pcapng"
** (Wireshark:48818) 10:36:40.901808 [Capture MESSAGE] -- Capture Stop ...
** (Wireshark:48818) 10:36:40.942369 [Capture MESSAGE] -- Capture stopped.
^C
```




```
(water@kali)~[/Desktop]
$ sudo wireshark -i lo -f 'tcp port 80' -w evidence
** (wireshark:48818) 10:31:59.154158 [GUI WARNING] -- Failed to register with host portal QDBusError("org.freedesktop.portal.Error.Failed", "Could not register app ID: Connection already associated with an application ID")
** (wireshark:48818) 10:32:18.206975 [Capture MESSAGE] -- Capture Start ...
** (wireshark:48818) 10:32:18.252094 [Capture MESSAGE] -- Error message from child: "The file to which the capture would be saved ('evidence') could not be opened: Is a directory.", ""
** (wireshark:48818) 10:32:20.720652 [Capture MESSAGE] -- Capture stopped.
** (wireshark:48818) 10:32:22.950501 [Capture MESSAGE] -- Capture Start ...
** (wireshark:48818) 10:32:22.987755 [Capture MESSAGE] -- Capture started
** (wireshark:48818) 10:32:22.987788 [Capture MESSAGE] -- File: "/tmp/wireshark_LoopbackEEAV3.pcapng"
** (wireshark:48818) 10:36:40.901808 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:48818) 10:36:40.942369 [Capture MESSAGE] -- Capture stopped.
^C
```

```
(water@kali)~[/Desktop]
$ sha256sum image_traffic.pcapng
e397a346297e0b615da2195d369354531d1cd75ba3c7e7fb4e92396d0dae3a3f image_traffic.pcapng
```

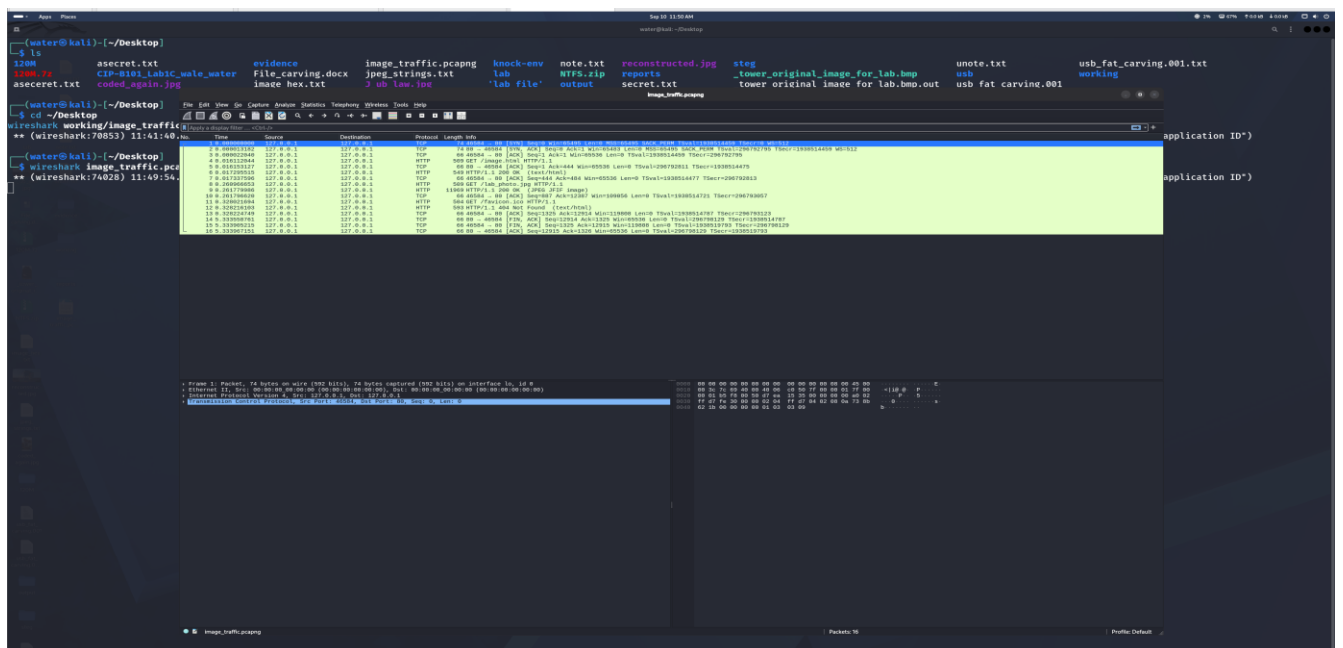
10. Part C - Prove That Two Objects Were Requested

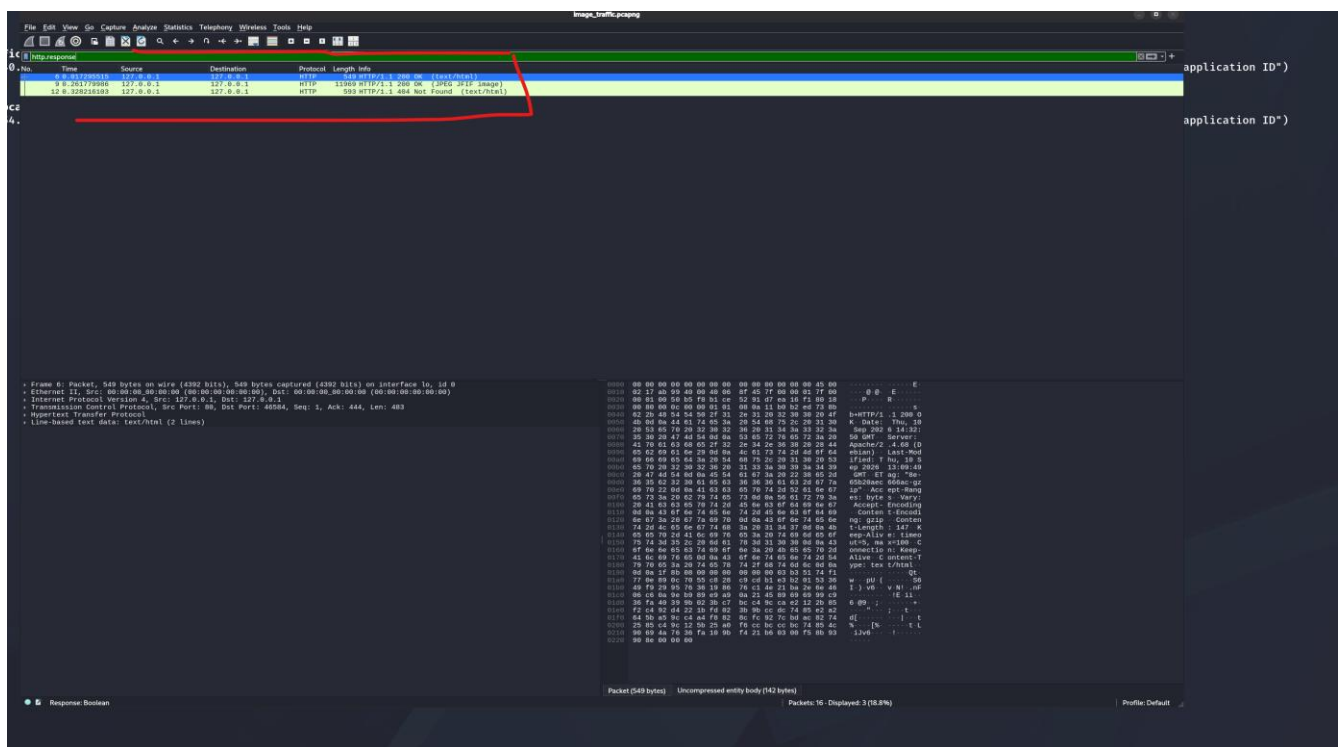
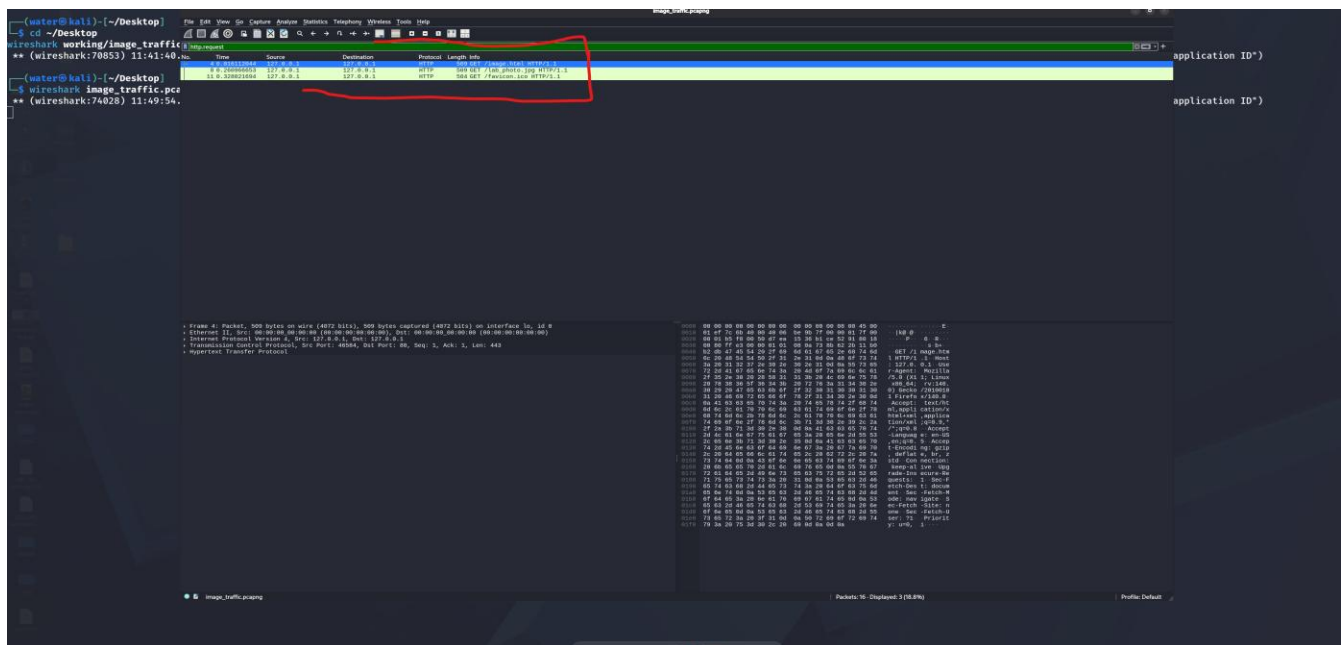
Apply http.request. You should normally see one GET request for image.html and another for lab_photo.jpg. A favicon or browser-generated request may also appear; document it rather than deleting it.

```
tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host \
  | tee reports/http_object_requests.tsv
```

```
tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e
http.content_length \
  | tee reports/http_object_responses.tsv
```

Wireshark was used instead of tshark for GET request for Image.html and lab_photo.jpg





11. Part D - Analyze TCP Segmentation and Reassembly

- Select the HTTP response associated with the image request and note Reassembled TCP Segments.
- Record the number and tcp.len value of each contributing segment.

7. Add the TCP payload lengths and compare the result with the HTTP header plus object data.

8. Explain why the exact segment sizes may differ from those in the slide: interface offloading, operating system buffers, MTU and capture location can change segmentation.

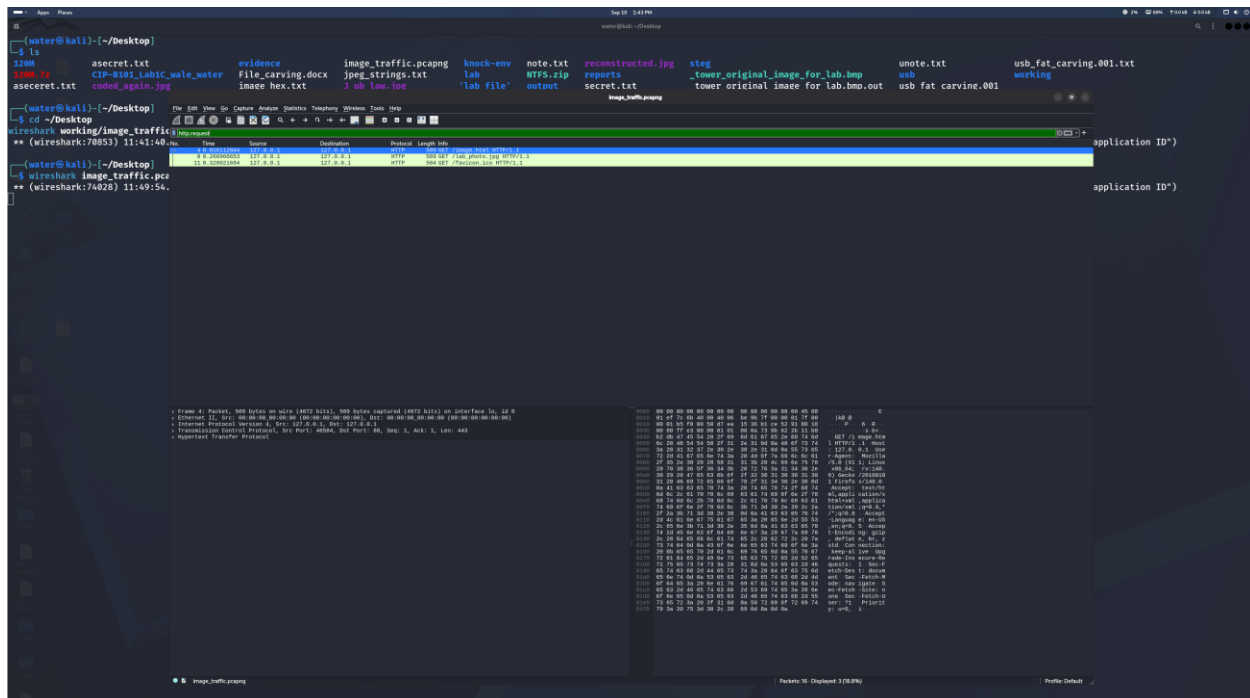
```
# Replace STREAM with the tcp.stream number for the image transfer
```

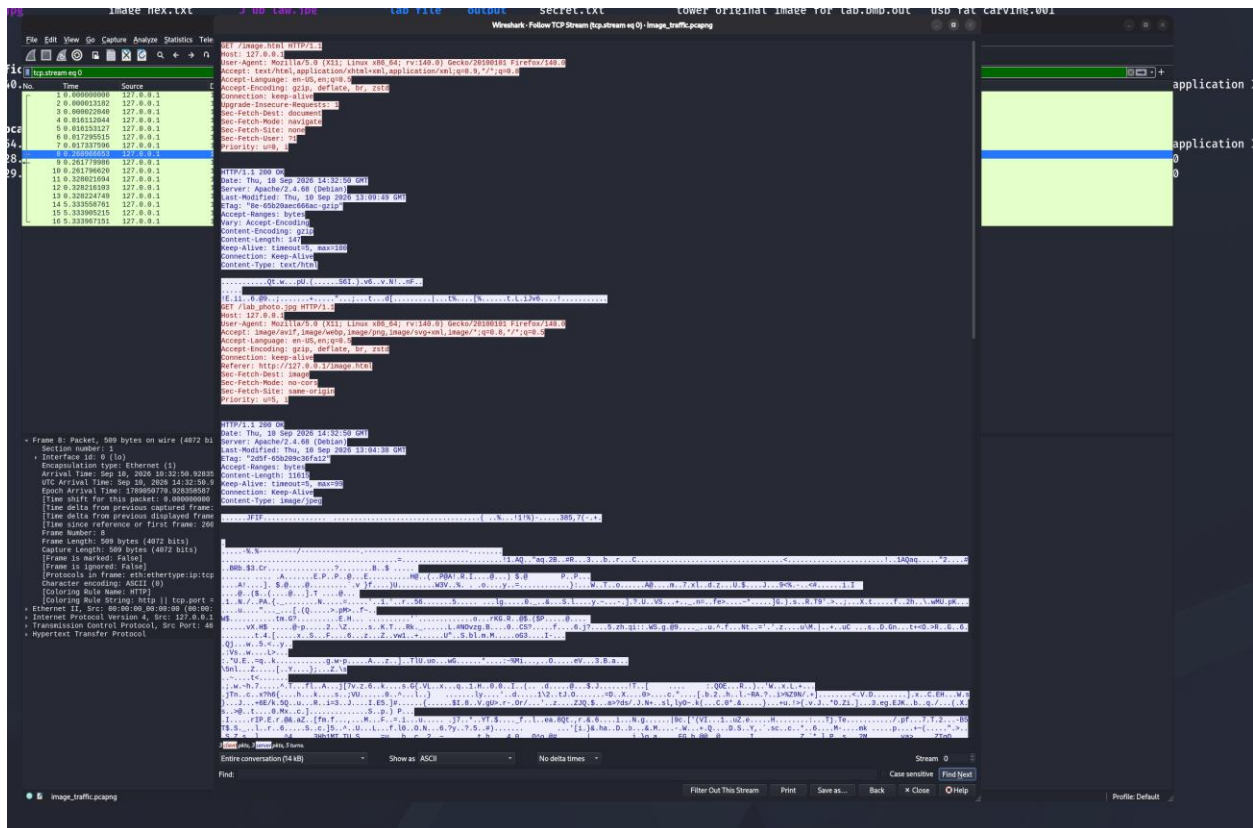
```
tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==STREAM && tcp.len>0' -T fields \
```

```
-e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
| tee reports/image_stream_segments.tsv
```

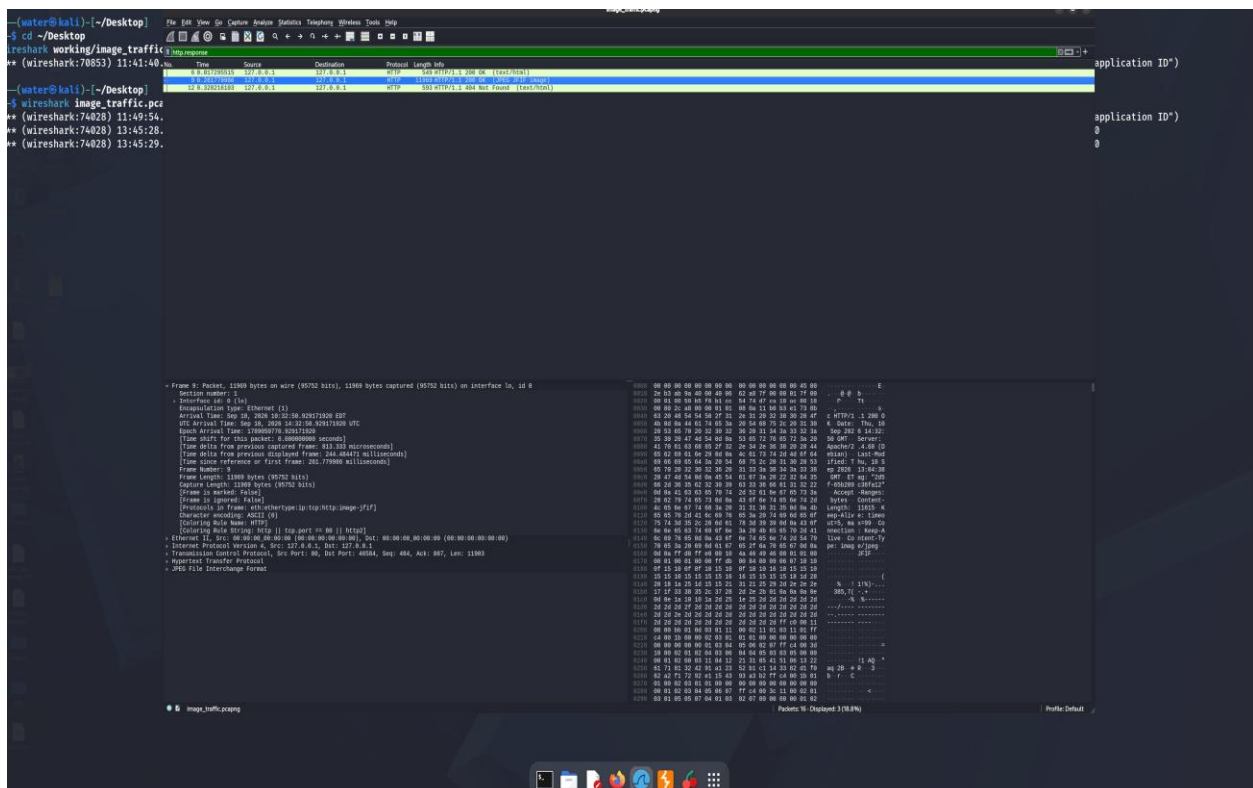
Summary of conversation sizes

```
tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee
reports/tcp_conversations.txt
```

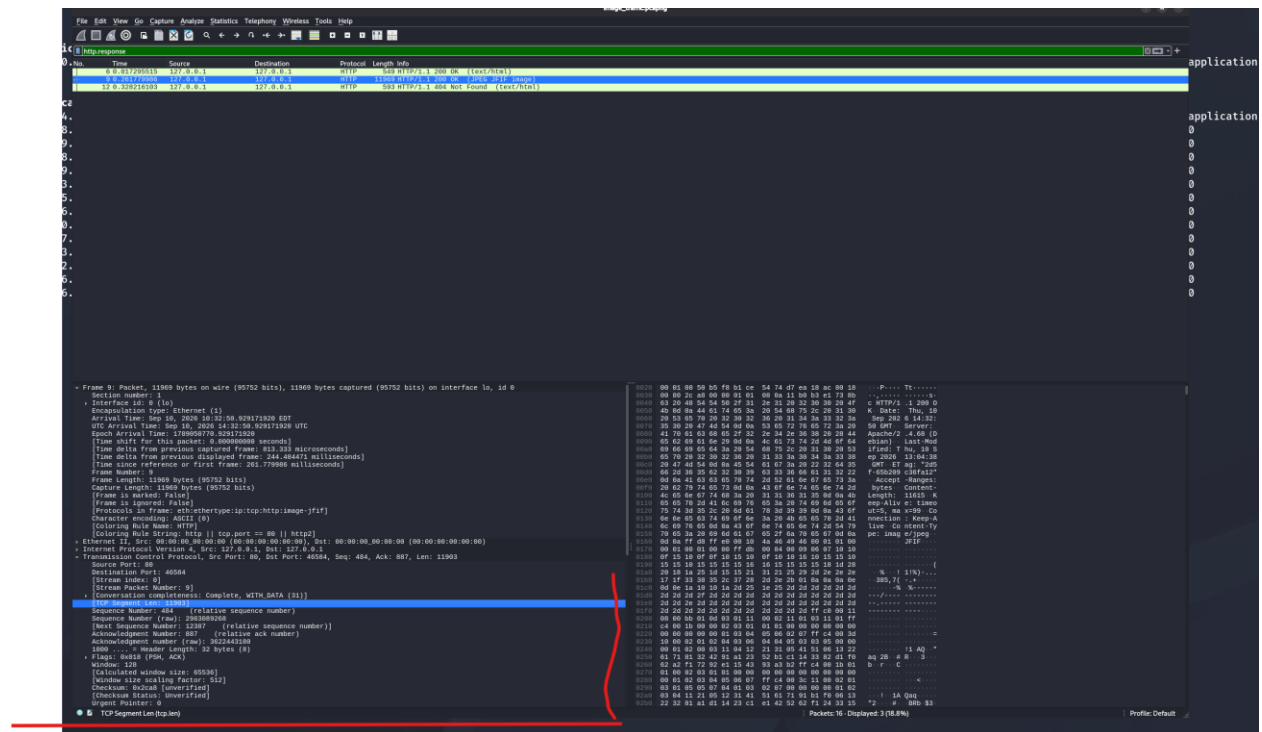




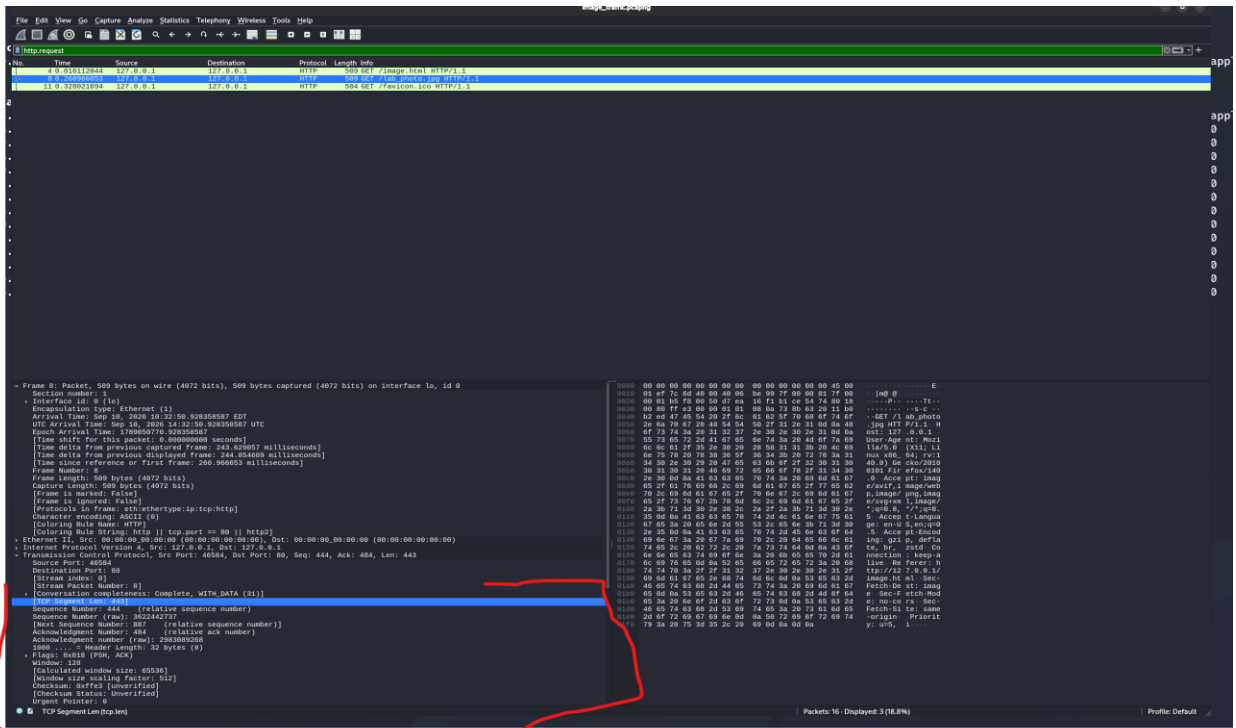
http.request and !(tcp.stream eq 0)



The TCP Lens Segment is 11903



The TCP Lens Segment is 443 for lab_photo.jpg



Transmission Control Protocol, Src Port: 46584, Dst Port: 80, Seq: 887, Ack: 12387, Len: 438

Explain why the exact segment sizes may differ from those in the slide: interface offloading, operating system buffers, MTU and capture location can change segmentation.

The exact TCP segment sizes may be different because network settings can affect how data is divided. Network interface offloading may combine or split packets before they are captured. The operating system buffers also control how much data is sent at a time. The MTU determines the maximum packet size allowed on the network. Finally, the location where the traffic is captured can affect the segment sizes seen in Wireshark.

12. Part E - Export the Embedded Image

9. In Wireshark select File > Export Objects > HTTP.
10. Locate lab_photo.jpg by hostname, content type or filename and save it under exported.
11. Alternatively, use tshark object export as shown below.
12. Compare file type, size and SHA-256 hash of the exported image with the source image.

```
mkdir -p exported/http_objects
```

```
tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
```

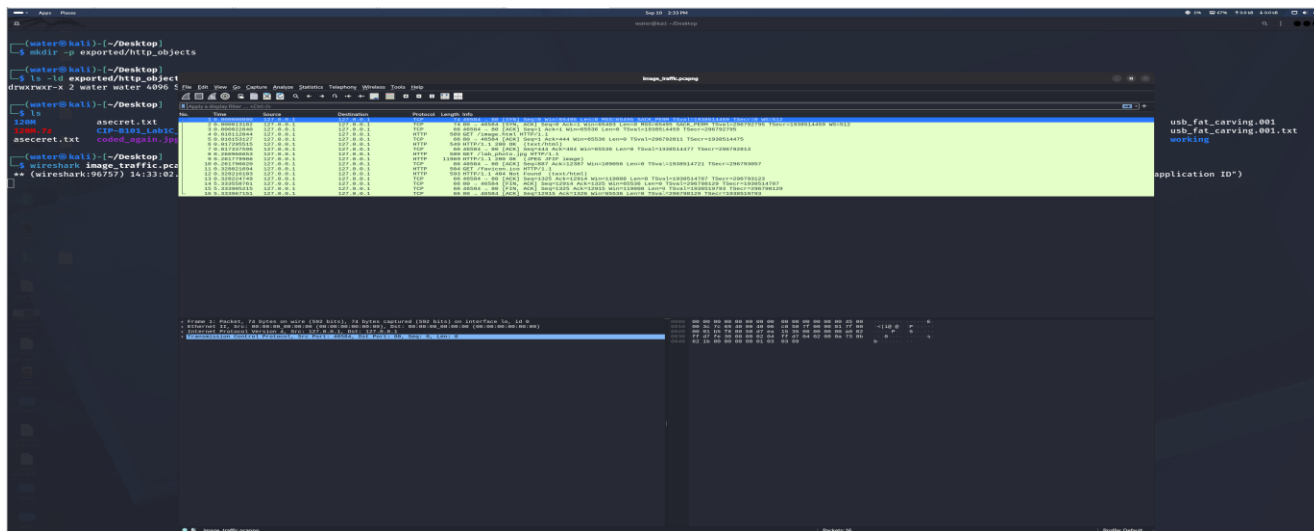
```
find exported/http_objects -maxdepth 1 -type f -ls
```

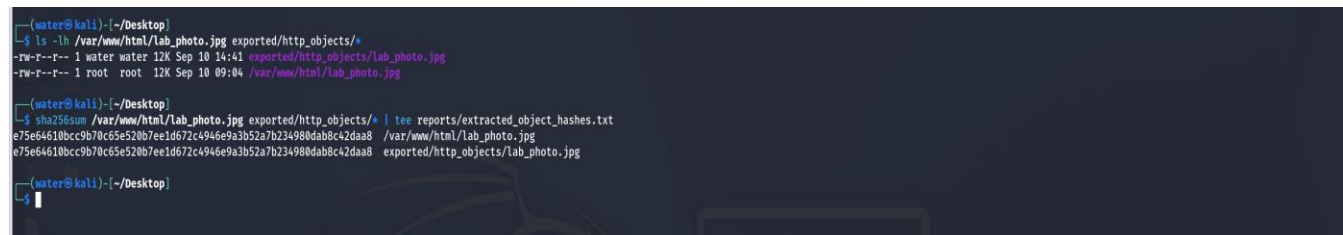
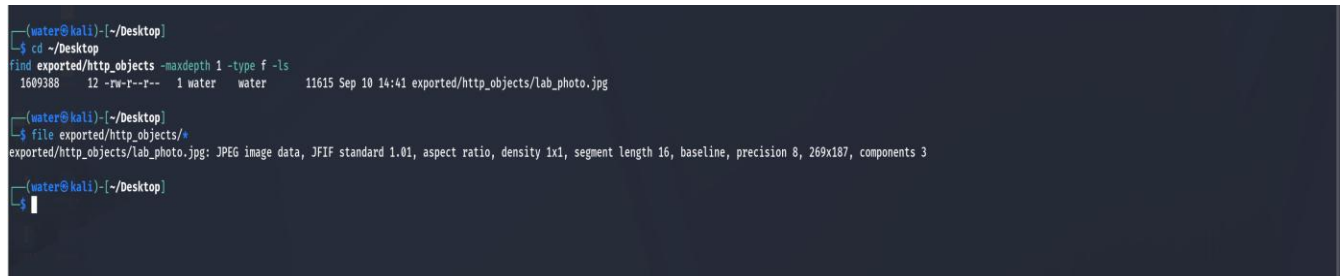
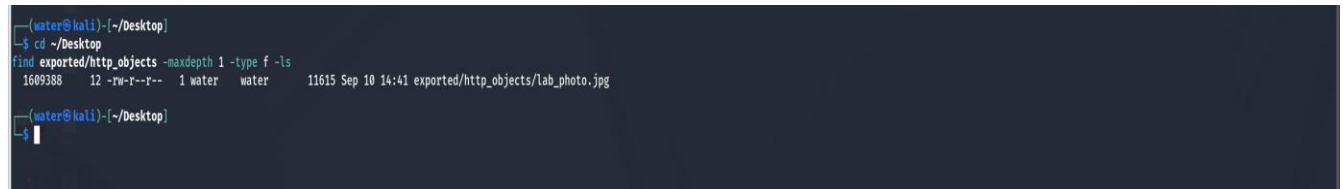
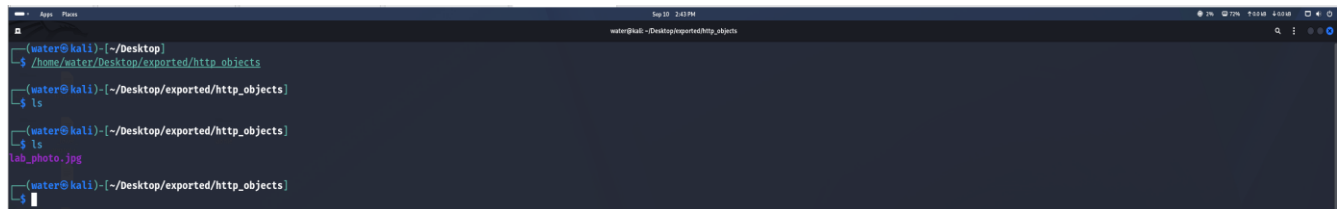
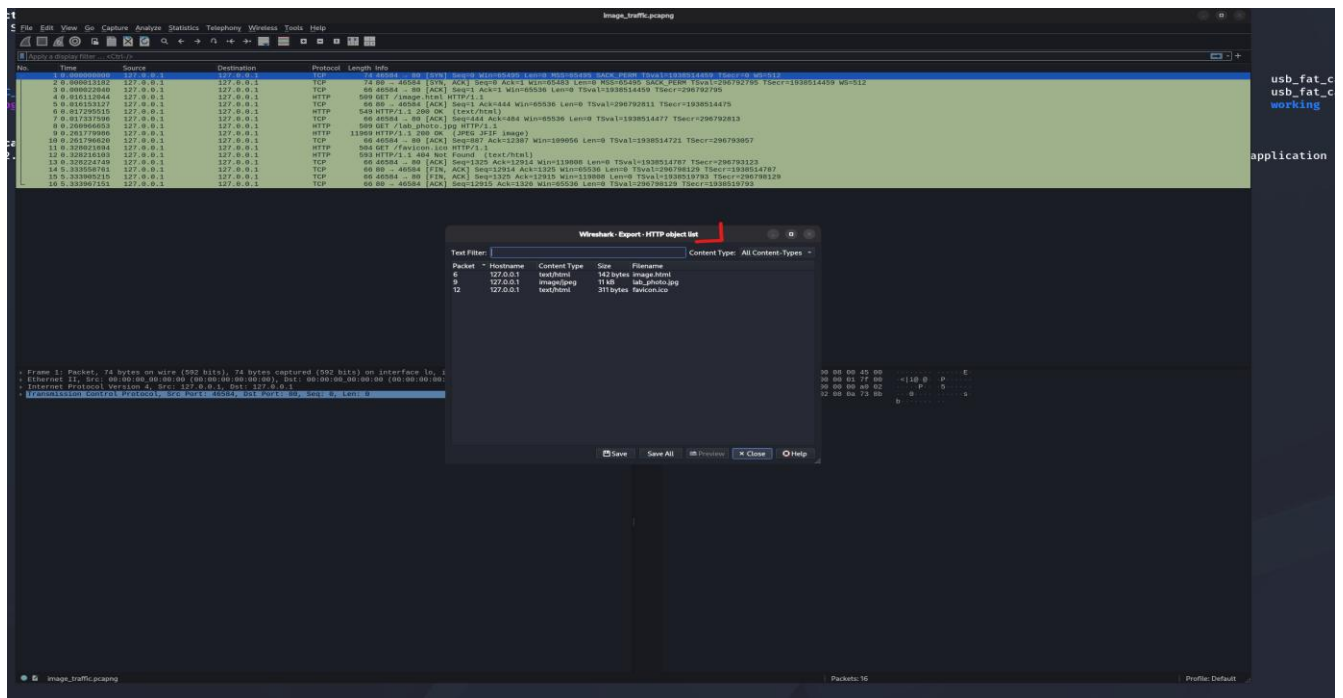
```
file exported/http_objects/*
```

```
sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee  
reports/extracted_object_hashes.txt
```

```
(water@kali) ~/Desktop
$ mkdir -p exported/http_objects
(water@kali) ~/Desktop
$ ls -ld exported/http_objects
drwxrwxr-x 2 water water 4096 Sep 10 14:30 exported/http_objects
(water@kali) ~/Desktop
```

```
(water@kali) ~/Desktop
$ mkdir -p exported/http_objects
(water@kali) ~/Desktop
$ ls -ld exported/http_objects
drwxrwxr-x 2 water water 4096 Sep 10 14:30 exported/http_objects
(water@kali) ~/Desktop
$ file exported/http_objects/*
exported/http_objects/lab_photo.jpg: JPEG image data, JFIF standard, Luminance & Chrominance
(water@kali) ~/Desktop
$ sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
/var/www/html/lab_photo.jpg 4b1d111111111111111111111111111111111111111111111111111111111111
exported/http_objects/lab_photo.jpg 4b1d111111111111111111111111111111111111111111111111111111111111
```





13. Part F - Compare curl and Browser Behaviour

Run curl against image.html in a new capture. By default, curl retrieves the HTML but does not automatically fetch the referenced image. Explain why the image request is absent unless you request it separately.

```
sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
sleep 2
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait
```

```
tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e http.request.uri | tee
reports/curl_requested_objects.txt
```

```
--(water@kali):~/Desktop
└─$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng &
[1] 105249

--(water@kali):~/Desktop
└─$ [sudo] password for water:

[1] * suspended (tty input) sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng
--(water@kali):~/Desktop
└─$ curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 36178
% Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0     0      0      0     0      0      0      0      0  0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 10 Sep 2026 19:00:04 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Thu, 10 Sep 2026 13:09:49 GMT
< ETag: "8e-65b20aec666ac"
< Accept-Ranges: bytes
< Content-Length: 142
< Vary: Accept-Encoding
< Content-Type: text/html
<
[142 bytes data]
100  142 100  142  0    0  3727    0
* Connection #0 to host 127.0.0.1:80 left intact

--(water@kali):~/Desktop
└─$ ls -lh /tmp/image_page.html
-rw-rw-r-- 1 water water 142 Sep 10 15:00 /tmp/image_page.html

--(water@kali):~/Desktop
└─$
```

Required Forensic Findings

Finding	Student Observation
Number of HTTP requests	3 HTTP requests were captured in the browser session (GET /image.html, GET /lab_photo.jpg, and GET /favicon.ico).
HTML request URI and timestamp	GET /image.html HTTP/1.1 – Frame 4, captured at 0.016112044s into the capture (relative time).
Image request URI	GET /lab_photo.jpg HTTP/1.1 – Frame 8, captured at 0.260966853s into the capture (relative time).

and timestamp	
HTTP response codes	image.html: HTTP/1.1 200 OK (Frame 6). lab_photo.jpg: HTTP/1.1 200 OK (Frame 9). favicon.ico: HTTP/1.1 404
Image content type	image/jpeg (confirmed by curl -I and by Wireshark identifying the object as JPEG JFIF image).
Reported Content-Length	image.html: 142 bytes. lab_photo.jpg: 11615 bytes. (favicon.ico response: 311 bytes.)
Number of reassembled TCP segments	1 segment. The entire HTTP response for lab_photo.jpg (headers + JPEG body) was carried in a single TCP segment (Frame 9), so no multi-segment reassembly was required on the loopback capture.
Sum of TCP payload lengths	11903 bytes (TCP Segment Len for Frame 9), which equals the 288-byte HTTP response header plus the 11615-byte JPEG body (288 + 11615 = 11903).
Original image SHA-256	e75e64610bcc9b70c65e520b7ee1d672c4946e9a3b52a7b234980dab8c42daa8 (/var/www/html/lab_photo.jpg)
Extracted image SHA-256	e75e64610bcc9b70c65e520b7ee1d672c4946e9a3b52a7b234980dab8c42daa8 (exported/http_objects/lab_photo.jpg)
Hash comparison conclusion	The SHA-256 hash of the exported/extracted image is identical to the original source image, confirming the object was carved from the capture without corruption or tampering.
curl/browser difference	The curl-only capture showed only one GET request (/image.html); curl does not parse HTML, so it never issued a follow-up request for lab_photo.jpg. A browser parses the HTML, finds the tag, and automatically issues a second GET request for the embedded image (plus a favicon request), producing multiple HTTP requests where curl produces just one.

2.7. Analysis and Findings

The analysis showed that the browser generated separate HTTP requests for image.html and the embedded lab_photo.jpg file. Wireshark was used to examine the requests, responses, TCP streams, and image transfer. The image response was successfully identified and the TCP segments carrying the image data were reviewed. The embedded image was exported from the HTTP traffic and compared with the original source image. SHA-256 hashes were used to check whether the extracted image matched the original image. The curl test showed that curl normally requested only the HTML page and did not automatically request the embedded image.

2.8. Challenges and Solutions

The challenge was getting TShark to save the packet capture because of a folder permission problem.

2.9. Conclusion

The lab demonstrated how a simple webpage can generate multiple HTTP requests and responses. The browser requested the HTML document and then separately requested the embedded image. Wireshark made it possible to observe these requests and examine the TCP segments carrying the image data. The image was successfully extracted from the captured HTTP traffic. Hash comparison provided a way to verify the integrity of the extracted evidence. The exercise also showed the difference between browser behavior and curl when handling embedded web resources.

2.10. Recommendations

An online class should be provided to explain the best ways to capture traffic using Wireshark. The class should include practical demonstrations.

2.11. References

Class Video Guide from Founder ICDFA on HTTP, TCP, Wireshark Network Forensics and TCP SYN Flood Analysis with Wireshark